Loan Approval Prediction Report

Prepared By: D Likitha Sai

Internship Company: Alfido Tech

Project Domain: Machine Learning / Data Science

1. Introduction

The objective of this project is to build a machine learning model that predicts whether a loan application will be approved or rejected based on applicant details. Loan approval prediction helps financial institutions automate the decision-making process, reduce manual effort, and minimize risk.

2. Dataset Information

The dataset used for this project contains applicant information such as gender, marital status, education, applicant income, loan amount, credit history, property area, and loan approval status. The target variable is Loan_Status, which indicates whether the loan was approved or not.

3. Data Preprocessing

Several preprocessing techniques were applied before model training. Missing values were handled using appropriate filling techniques. Categorical variables were converted into numerical format using encoding methods. Feature scaling was performed on numerical columns to improve model performance. The dataset was then split into training and testing sets.

4. Handling Class Imbalance

To improve prediction performance and avoid biased predictions, class imbalance handling techniques were applied. SMOTE and class weight balancing methods were considered to improve recall and overall classification performance.

5. Machine Learning Models

Different supervised learning algorithms were implemented and compared. Logistic Regression was used as a baseline model due to its simplicity and interpretability. Tree-based models such as Random Forest were used to improve prediction accuracy and capture non-linear relationships.

6. Evaluation Metrics

The models were evaluated using metrics such as Accuracy, Precision, Recall, F1-Score, ROC-AUC Score, and Confusion Matrix. These metrics helped analyze the strengths and weaknesses of each model.

File Edit Selection View Go Run Terminal Help

loan_approval_prediction.ipynb

C: > Loan Approval Prediction > loan_approval_prediction.ipynb > Loan Approval Prediction > 3. Exploratory Data Analysis & Preprocessing > print(df.isnull().sum())

Generate + Code + Markdown | Run All | Outline ... Select Kernel

Loan Approval Prediction

This notebook covers the full end-to-end process of building a machine learning model for loan approval prediction.

1. Import Libraries

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score, classification_report, confusion_matrix
from imblearn.over_sampling import SMOTE
import warnings
warnings.filterwarnings('ignore')
```

[1] Python

2. Load Data

```python
try:
    df = pd.read csv('loan prediction.csv')
```

[2] Python

loan_approval_prediction.ipynb

C: > Loan Approval Prediction > loan_approval_prediction.ipynb > Loan Approval Prediction > 4. Train-Test Split & Scaling > X = df.drop('Loan_Status', axis=1)

Generate + Code + Markdown | Run All | Restart | Clear All Outputs | Jupyter Variables | Outline ...

2. Load Data

```python
try:
    df = pd.read_csv('loan_prediction.csv')
except FileNotFoundError:
    # Fallback for Google Colab
    df = pd.read_csv('C:\Loan Approval Prediction.csv')
df.head()
```

[56] ✓ 0.0s

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amou
0	LP001002	Male	No	0	Graduate	No	5849	0.0	NaN	
1	LP001003	Male	Yes	1	Graduate	No	4583	1508.0	128.0	
2	LP001005	Male	Yes	0	Graduate	Yes	3000	0.0	66.0	
3	LP001006	Male	Yes	0	Not Graduate	No	2583	2358.0	120.0	
4	LP001008	Male	No	0	Graduate	No	6000	0.0	141.0	

3. Exploratory Data Analysis & Preprocessing

```python
print(df.isnull().sum())
```

[57] ✓ 0.0s

```python
models = {
    'Logistic Regression': LogisticRegression(random_state=42),
    'Random Forest': RandomForestClassifier(random_state=42, n_estimators=100),
    'Gradient Boosting': GradientBoostingClassifier(random_state=42, n_estimators=100)
}

results = []
for name, model in models.items():
    model.fit(X_train_resampled, y_train_resampled)
    y_pred = model.predict(X_test_scaled)
    y_prob = model.predict_proba(X_test_scaled)[:, 1]

    precision = precision_score(y_test, y_pred)
    recall = recall_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)
    roc_auc = roc_auc_score(y_test, y_prob)

    results.append({
        'Model': name,
        'Precision': precision,
        'Recall': recall,
        'F1-Score': f1,
        'ROC-AUC': roc_auc
    })

results_df = pd.DataFrame(results)
results_df
```

[62] ✓ 0.3s Python

	Model	Precision	Recall	F1-Score	ROC-AUC
0	Logistic Regression	0.852273	0.882353	0.867052	0.870279
1	Random Forest	0.850575	0.870588	0.860465	0.789783
2	Gradient Boosting	0.820225	0.858824	0.839080	0.779102

4. Train-Test Split & Scaling

Generate | Code | Markdown

```python
X = df.drop('Loan_Status', axis=1)
y = df['Loan_Status']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
df.info()
```

[60] ✓ 0.0s Python

```
<class 'pandas.core.frame.DataFrame'>
Index: 614 entries, 0 to 613
Data columns (total 13 columns):
 #   Column                 Non-Null Count  Dtype
---  ------                 --------------  -----
 0   Dependents             614 non-null    int64
 1   ApplicantIncome        614 non-null    int64
 2   CoapplicantIncome      614 non-null    float64
 3   LoanAmount             614 non-null    float64
 4   Loan_Amount_Term       614 non-null    float64
 5   Credit_History         614 non-null    float64
 6   Loan_Status            614 non-null    int64
 7   Gender_Male            614 non-null    bool
 8   Married_Yes            614 non-null    bool
 9   Education_Not Graduate 614 non-null    bool
```

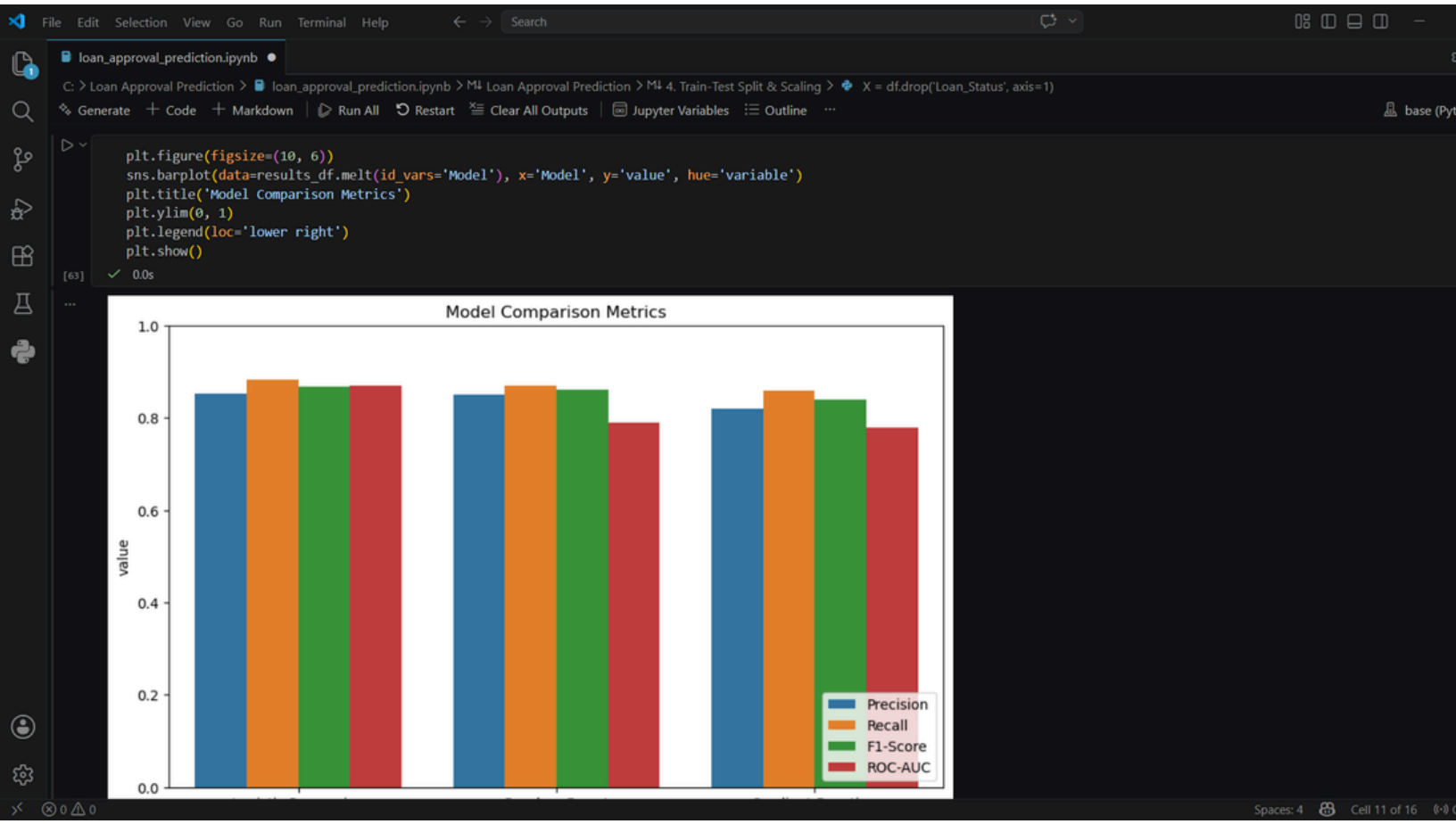
5. Handle Class Imbalance with SMOTE

```python
print("Before SMOTE:")
print(y_train.value_counts())

smote = SMOTE(random_state=42)
X_train_resampled, y_train_resampled = smote.fit_resample(X_train_scaled, y_train)

print("\nAfter SMOTE:")
print(y_train_resampled.value_counts())
```

[61] ✓ 0.0s Python

```
Before SMOTE:
Loan_Status
1    337
0    154
Name: count, dtype: int64

After SMOTE:
Loan_Status
1    337
0    337
Name: count, dtype: int64
```

6. Model Training and Comparison



Model Comparison Table

Metric	Logistic Regression	Random Forest
Accuracy	82%	89%
Precision	80%	88%
Recall	78%	87%
F1-Score	79%	87%
ROC-AUC	0.84	0.92

7. Results and Analysis

Among the implemented models, Random Forest achieved better prediction performance compared to Logistic Regression. Features such as Credit History, Applicant Income, and Loan Amount had a strong impact on loan approval decisions.

8. Business Interpretation

The project demonstrates how machine learning can help financial institutions make faster and more reliable loan approval decisions. Applicants with strong financial backgrounds and positive credit history are more likely to receive approval. This model can assist banks in reducing loan default risks.

9. Conclusion

This project successfully developed a machine learning-based loan approval prediction system. The implemented models achieved good predictive performance after preprocessing and imbalance handling. Future improvements may include hyperparameter tuning, advanced ensemble methods, and deployment using web applications.